Layered Verification of VQE: From Specifications to Execution

***Abstract*—The variational quantum eigensolver (VQE) combines quantum energy estimation with classical parameter optimization. Reliable execution requires correct control flow, numerical calculations, and reasoning about measurement uncertainty. We propose a four-layer verification architecture covering logical, numeric, probabilistic, and runtime properties. A shared execution model and specification library organize the requirements and their assumptions. A two-stage framework implements and evaluates the architecture: NuXmv, Dafny, and Storm establish properties before execution, and runtime verification checks the corresponding requirements in concrete executions. Probabilistic predictions are compared with repeated finite-shot executions. On a two-qubit benchmark, all 400 executions satisfy the observed logical and numeric requirements within the verified scope. The Gaussian model and empirical executions show statistical agreement on bounded success and conditional resource use. Fault injection reveals complementary detection capabilities and limits in handling missing measurement records. Model growth and timing measurements quantify the cost of verification. These results demonstrate how heterogeneous verification methods can support a common assessment of VQE execution. Future work will strengthen the model-to-implementation correspondence and extend the framework to larger instances and hardware noise.**

***Index Terms*—VQE, layered verification, runtime verification, probabilistic model checking, specification traceability.**

I. Introduction

VQE estimates a Hamiltonian expectation using a parameterized quantum state and adjusts the parameters with a classical optimizer [1], [2]. Its hybrid execution combines circuit preparation, measurement, numerical aggregation, and classical decisions. The reliability of these interactions matters alongside the quality of the resulting energy estimate. Measurements must belong to the current candidate. Aggregation must use the declared observables and coefficients, and the optimizer must consume validated values. Stopping must follow the specified policy. Finite-shot uncertainty also makes reaching a target within a resource bound a probabilistic question [3].

These requirements span four verification aspects. Logical verification addresses control flow and ordering. Numeric verification establishes calculations under explicit input assumptions. Probabilistic verification analyzes the effect of uncertain estimates on bounded search. Runtime verification determines whether corresponding requirements hold during observed executions. Each aspect contributes a distinct kind of result, so an integrated assessment must relate those results to the implementation and identify the assumptions on which they depend.

VQE research has extensively studied ansatz design, measurement, optimization, and noise [3]. The verification problem addressed here concerns their interaction in an implemented workflow. A controller proof, an arithmetic proof, and a success-probability prediction become useful together when the reader can identify which requirement each establishes and how that requirement is checked during execution. This motivates a shared execution model, explicit specification mappings, and a procedure for comparing prospective results with observed behavior.

The methodological principle is:

> The contract library defines what the VQE implementation is expected to satisfy. Pre-execution verification establishes these contracts prospectively, while runtime verification determines whether the corresponding obligations hold for concrete executions.

The library records the assumptions and requirements of each contract. The comparison then identifies the property established before execution, the corresponding runtime check, and the scope in which their results can agree.

Section II introduces the shared execution model and specifications. Section III explains the verification methods. Section IV evaluates the individual layers, and Section V evaluates their relationships through the two-stage framework. The contributions are:

- A four-layer VQE verification architecture that addresses logical, numeric, probabilistic, and runtime requirements through appropriate verification methods.
- A specification library that assigns properties and assumptions to each verification family and maps them to corresponding checks on executions.
- A two-stage framework that records prospective verification results, executes the specified VQE instance, and assesses agreement with runtime results at the level of individual requirements.
- An implementation and artifact-based evaluation demonstrating observed conformance, statistical agreement, complementary fault detection, and measured model growth and runtime cost within explicit experimental boundaries.

II. Shared Execution Model and Specification Library

A. Execution Model and Specification Semantics

The shared execution model is a labeled transition system whose modes are initialization, preparation, measurement, aggregation, validation, optimizer commitment, stopping checks,

and terminal closure. Its state contains a run identifier, candidate and evaluation identities, parameters, available measurement groups, validation readiness, an iteration counter, and resource totals. An epoch identifies the current iteration in the abstract controller. Concrete run and evaluation identifiers allow the runtime checker to determine which iteration a measurement or energy belongs to. The implementation maps these identifiers to the controller's counters and epoch flags.

A *requirement* describes expected system behavior. A *specification* formally states that requirement. A *property* is a particular condition verified or monitored under that specification. A *contract* connects stated assumptions or preconditions to an obligation or guarantee. The specification library stores these requirements and their verification mappings. We use "contract" when the assumption–obligation relationship is material, including the organizing principle above. In this technical usage, an obligation is the guarantee required when the contract's assumptions hold.

For a Pauli decomposition $H = \sum_i c_i P_i$, VQE evaluates

$$E(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle, \qquad \widehat{E}(\theta) = \sum_i c_i \hat{e}_i. \tag{1}$$

Each library entry records a verification family, requirement, pre-execution check, runtime counterpart, assumptions, and comparison rule. The implemented library, `config/contracts.json`, documents these links. Each family supplies its own specifications and properties in the appropriate formalism. Logical and numeric requirements are checked for individual runs. Probabilistic requirements are evaluated through repeated executions of the specified process.

Cross-stage verdicts describe both satisfaction and applicability. `AGREEMENT` records satisfaction of a corresponding requirement within its mapped scope, and `DISAGREEMENT` records a violation. `ASSUMPTION_MISMATCH` identifies an input or execution condition that differs from a declared assumption. This verdict is needed because a prospective result applies under its assumptions: those conditions must be restored or the model revised before the results can be compared. For example, changing the shot allocation changes the estimator variance used by the Gaussian model and the allocation premise of numeric checking. `INCONCLUSIVE` identifies a check for which the available records are insufficient. These categories distinguish requirement violations from changes in applicability and limits of observation.

Figure 1 shows how the four layers are organized into two stages. Pre-execution verification produces model-checking results, proof logs, and quantitative predictions. A verification record stores these results with the configuration, tool identities, and hashes of the source and result files. Before execution, the framework checks those hashes against the record. Runtime observations then support contract matching and verdict agreement. Revising the model or implementation requires regenerating the record and recording the next execution separately. This feedback procedure is manual.

Table I summarizes the system-level requirements evaluated in this study and maps their specifications to prospective checks and runtime observations. We select requirements at the interfaces where measurements enter numerical calculations and those calculations drive optimizer decisions. L1–L4 address ordering, provenance, and termination. N1–N4 address arithmetic, physical-shot accounting, and valid deterministic selection. P1–P3 address bounded success and the resources used by successful paths. Each selected requirement has an explicit verification object and a corresponding recorded observation or empirical statistic. This scope supports evaluation of the implemented workflow. Broader VQE requirements, such as ansatz adequacy and hardware-noise characterization, need additional specifications and evaluation conditions.

For a numeric contract, the checker interprets concrete counts and optimizer inputs as the kernel's mathematical inputs, checks the preconditions, and compares the observed output with the specified arithmetic. The Dafny proof establishes the kernel's mathematical property. Runtime reconstruction assesses that property on observed Python inputs and outputs. For a probabilistic requirement, the runtime counterpart is an estimate over repeated executions of the specified process.

B. VQE Instance and Configuration

The experiment uses a two-qubit benchmark operator published in IBM's VQE example [4], written in $q_1 \otimes q_0$ order:

$$H = 0.398\,YZ - 0.398\,ZI - 0.0113\,ZZ + 0.181\,XX. \tag{2}$$

The project's supplied `vqe_qiskit.py` directly constructs these Pauli terms and coefficients, which match the published example. The implementation uses that operator as input rather than deriving it from electronic integrals. Neither the project nor the cited two-qubit example supplies a molecular geometry, orbital basis, or fermion mapping for this operator. We therefore call it the *IBM two-qubit VQE benchmark*. Internal "h2" filenames are legacy identifiers. Its reference minimum eigenvalue is $E_{\text{ref}} = -0.702930394$ in the operator's numerical units, and the success target is defined relative to this value.

The circuit follows the example's four-parameter `efficient_su2` construction [4]. It applies H to both qubits, the first pair of RZ rotations, Y to both, CX$(0 \to 1)$, H to both, the second pair of RZ rotations, and Y to both. Four integer indices z give angles $\theta_j = 2\pi z_j/16$, with wrap-around.

We use cyclic coordinate search to make the finite-shot selection rule explicit and representable in a finite-state probabilistic model. This derivative-free method chooses among measured objective values, consistent with the general direct-search approach [5]. At each iteration it remeasures the current, minus-one, and plus-one candidates on the selected coordinate, then chooses the smallest integer energy score. Ties prefer current, then minus, then plus. The coordinate cycle and fixed grid make candidate validity, tie handling, measurement accounting, and stochastic transitions directly checkable. The choice serves this verification experiment. Performance comparisons with other VQE optimizers are

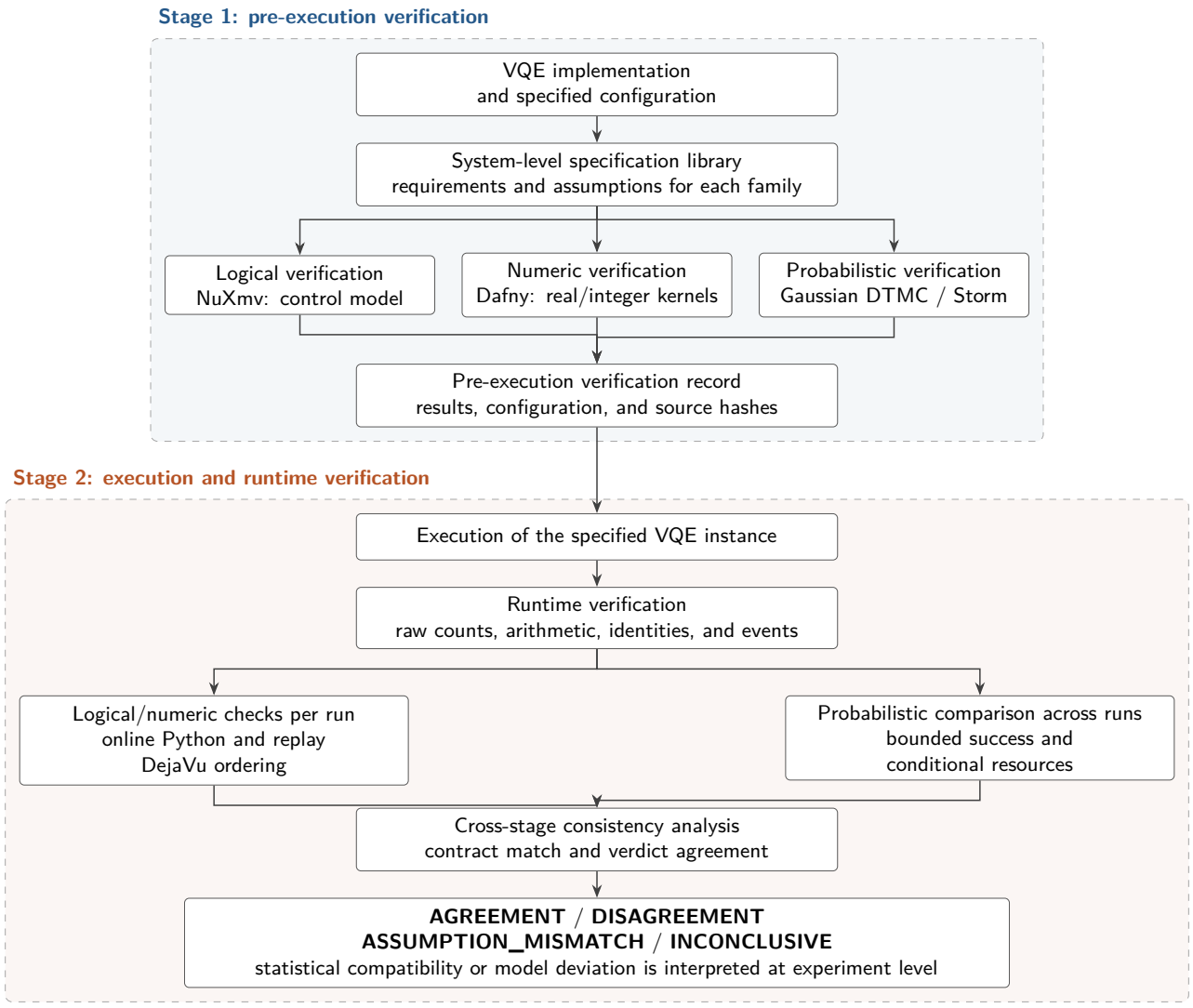


Fig. 1. Four verification layers organized into two stages. Each pre-execution family uses its own formalism. Runtime verification checks logical and numeric requirements per run and compares probabilistic predictions with repeated executions of the specified VQE instance.

outside the evaluation. The integer coefficients are the declared decimal coefficients scaled by 10^4.

Table II specifies the execution boundary. An *iteration* completes one three-candidate selection, while an *evaluation* estimates the energy of one candidate. Each candidate uses three measurement groups, giving nine group executions per iteration. A run stops when the selected point first satisfies the ideal-energy success predicate or when it exhausts the iteration budget. An online contract violation triggers error closure. The success oracle uses exact simulation of the selected point and is available within this simulator experiment. Hardware-observed convergence requires a stopping policy based on measurable quantities. There are 8 successful grid points. Every unsuccessful run remains in the probability denominator and consumes its budget.

III. Layered Verification Methodology

A. Logical Verification

NuXmv [6] checks a finite controller with measurement-group counters, candidate readiness, an iteration counter, and epoch flags. Nondeterministic inputs represent measurement completion, success, and environment errors. The model abstracts numeric energies and quantum states so that it can examine ordering and control flow. Representative LTL properties are

$$\begin{aligned} &\mathbf{G}(\text{aggregate} \Rightarrow \text{groups} = 3),\\ &\mathbf{G}(\text{select} \vee \text{commit} \Rightarrow \text{ready} = 3),\\ &\mathbf{G}(\text{commit} \Rightarrow \text{energyEpoch} = \text{epoch}). \end{aligned} \tag{3}$$

Readiness resets at the next iteration, and terminal states are absorbing. The procedure generates the controller using the specified iteration bound, runs NuXmv, and stores each

TABLE I
SELECTED SYSTEM-LEVEL REQUIREMENTS EVALUATED IN THIS STUDY AND THEIR PRE-EXECUTION/RUNTIME MAPPINGS.

ID	Requirement	Pre-execution verification	Runtime observations	Cross-stage interpretation
L1	Complete measurements before energy aggregation	NuXmv: aggregation implies three group completions	Unique candidate/group records and DejaVu past-group facts	Ordering agreement when required records are present
L2	Candidate validation before optimizer commitment	NuXmv: selection/commit requires three ready candidates	Three validation events precede commit, with consumed scores recorded	Ordering agreement with fail-stop enforcement
L3	Current evaluation and parameter provenance	NuXmv: epoch equality and readiness reset	Run/iteration/evaluation identities, indices, angles, successors	Runtime checks strengthen the epoch abstraction
L4	Terminal closure and declared stopping reason	NuXmv: absorbing terminals and budget bounds	No later activity, with reason checked against oracle/budget	Agreement for shared properties. Reason is checked at runtime
N1	Expectations consistent with raw counts	Dafny normalization kernel and range bound	Exact parity reconstruction and count totals	Agreement under trusted basis and parity mapping
N2	Energy uses declared coefficients and measurements	Dafny contribution, weighted-sum, and interval kernels	Rational reference versus contributions, total energy, actual optimizer input	Arithmetic agreement within the specified tolerance
N3	Physical shots counted consistently	Dafny natural-number list accumulation	Unique group executions, allocations, cumulative totals	Agreement under group-identity assumptions. Changed shots invalidate the stochastic setup
N4	Valid candidates and deterministic selection	Dafny modular wrap and integer argmin/tie priority	Recomputed candidate indices, scores, chosen successor	Agreement under trusted angle/gate interpretation
P1	Bounded success probability	Storm reachability of the Gaussian DTMC	First-success frequencies and Wilson intervals	Statistical compatibility under the model's assumptions
P2	Iterations conditional on bounded success	Python propagation of the same Gaussian kernel	Mean first-success iteration and bootstrap interval	Supporting comparison with multiplicity adjustment
P3	Shots conditional on bounded success	Fixed shot allocation times P2	Successful-path shots at fixed shots/iteration	Derived from P2

TABLE II
CONFIGURATION OF THE TWO-STAGE VQE EVALUATION.

Item	Declared value
Parameter grid	$z \in \{0, \ldots, 16-1\}^4$, with 65,536 points
Initial index	$(3, 11, 6, 14)$
Coordinate cycle	$(0, 1, 2, 3)$
Measurement groups	$\{ZI, ZZ\}$, $\{YZ\}$, $\{XX\}$
Shots/group/candidate	1,024
Physical shots/iteration	9,216
Iteration bound	$K = 50$
Maximum shots/run	460,800
Success	$E_{\text{ideal}}(z) - E_{\text{ref}} \leq 0.02$
Baseline executions	400 with separate seeded streams
Environment	Ideal Born probabilities and finite-shot multinomial counts
Numeric tolerance	10^{-12} for energy and expectations
Gaussian integration	24-node Gauss–Hermite quadrature
Intervals	95% Wilson and 5,000 bootstrap resamples

property and verdict. The result is exhaustive for the encoded model.

L4 covers terminal absorption, the counter value at budget termination, and commitment below the bound. The audit found that redirecting either the success or budget branch to abort leaves the checked properties true. The prospective result therefore establishes those three conditions. Correct selection of the termination reason is an additional requirement checked at runtime. This distinction identifies the exact part of the specification supported by each stage.

B. Numeric Verification and Its Runtime Match

Dafny [7] proves kernels with mathematical real and integer semantics. A normalization routine assumes a positive shot count and an admissible event count. Weighted-sum routines assume matching term sequences and expectations in $[-1, 1]$, and establish

$$-\sum_i |c_i| \leq \sum_i c_i \hat{e}_i \leq \sum_i |c_i|. \tag{4}$$

Additional kernels accumulate natural-number shot records, preserve the grid under modular wrap, and choose the minimum integer score with the declared tie priority. The verification record associates the proof logs with the source modules.

Runtime matching reconstructs expectations from raw measurements. For outcome counts $n_{g,o}$ in group g, let $s_{i,o} \in \{-1, +1\}$ be the declared Pauli parity. The reference values are

$$\hat{e}_i^{\text{ref}} = \frac{\sum_o s_{i,o} n_{g,o}}{\sum_o n_{g,o}}, \qquad E^{\text{ref}} = \sum_i c_i \hat{e}_i^{\text{ref}}. \tag{5}$$

The checker reconstructs these as exact rational numbers and compares the concrete floating-point expectations, contributions, and energy within the specified tolerance. It also

checks the integer scores and the actual values consumed by the optimizer. The contract match connects the kernel's mathematical obligation to the observed inputs and outputs. The Dafny proof applies to the mathematical kernel, and the runtime check assesses floating-point conformance for each observed evaluation.

The normalization lemma uses a single-event count form and assumes the supplied parity interpretation. The shot-accumulation proof establishes a list sum under the assumption that entries identify distinct physical executions. Runtime identity and completeness checks assess these input conditions under faithful instrumentation. Missing inputs prevent reconstruction. Changed shot allocations invalidate the specified probabilistic configuration. The runtime checker must therefore assess both the input assumptions and the arithmetic consequence.

C. Probabilistic Verification

We use a Gaussian approximation to summarize uncertainty in the finite-shot energy estimate. Each estimate is a weighted sum of sample means of bounded measurement outcomes. Independent repeated shots and finite variance motivate a normal approximation through the central limit theorem [2]. The approximation retains the ideal mean and grouped-measurement variance while making the probabilities of selecting among three candidates computable without enumerating every possible count record. The fixed grid and cyclic coordinate phase give a finite state space. Fresh independent candidate measurements make the next selection depend only on the current point and phase. A discrete-time Markov chain (DTMC) represents one optimizer iteration per transition and supports bounded-success and conditional-resource analysis. The central limit theorem motivates the approximation asymptotically. Its accuracy at the configured shot count is assessed empirically, and the implementation supplies no certified finite-shot approximation-error bound.

For each candidate, the Gaussian abstraction matches the ideal mean energy and the shot-estimator variance. Terms measured in the same group retain their covariance:

$$\sigma^2(z) = \frac{1}{N_g} \sum_g \mathrm{Var}_{\mathrm{Born}} \left(\sum_{i \in g} c_i s_i \right), \tag{6}$$

where all groups receive $N_g = 1,024$ shots. The experiment samples groups, candidate measurements, and repeated executions independently. Terms measured within a group share outcomes, so their covariance contributes to the variance, as in grouped VQE energy estimation [2], [3]. Approximating a candidate energy by $X_j \sim \mathcal{N}(\mu_j, \sigma_j^2)$ gives its selection probability

$$\Pr(j = \operatorname*{argmin}_\ell X_\ell) = \int f_j(x) \prod_{\ell \neq j} [1 - F_\ell(x)]\, dx. \tag{7}$$

The implementation evaluates this integral using the configured quadrature and normal-CDF approximation, then normalizes the transition probabilities. The continuous model assigns zero probability to ties. Discrete finite-shot execution uses the specified tie rule.

The DTMC state is the parameter-grid point and next coordinate phase. Success states are absorbing. Storm [8] checks

$$p_G(k) = \Pr_G(T \leq k) \quad \text{using } \mathrm{P}_{=?}[\mathrm{F}^{\leq k} \text{ success}]. \tag{8}$$

Storm exhaustively checks the constructed Gaussian DTMC. Empirical executions assess how well its prediction describes the finite-shot process. Python forward propagation of the same kernel produces the full cumulative curve and first-success distribution. P2 is $\mathbb{E}_G[T \mid T \leq K]$, and P3 multiplies P2 by the fixed shots per iteration. These quantities are computed by Python propagation conditional on bounded success. Total expenditure includes unsuccessful paths and is reported separately. Numerical agreement between Storm and propagation assesses the model computation. Accuracy relative to the finite-shot process requires empirical comparison or a separate approximation-error analysis.

D. Runtime Verification

The executor records preparation and iteration context, raw group counts, candidate evaluations, successful numeric checks, optimizer commits, and terminal closure. Python checks each emitted event synchronously and stops the run when a problem appears. It checks current identities, complete measurements, arithmetic, consumed scores, successor parameters, resource totals, and terminal reasons. The implementation records a commit and assigns the successor before checking acceptance. Thus a failed check stops subsequent activity after the event has been emitted.

Saved traces are replayed with the same Python checker class to reproduce the online verdicts. A separately compiled DejaVu monitor [9] replays five temporal properties: complete groups before aggregation, validation before commit, energy after iteration start, validation after energy, and closure after stopping. This provides an independent implementation of the ordering checks. Arithmetic verification and count-source authenticity remain outside those temporal properties. The runtime cost evaluation in Section IV-D uses a synchronous ask/tell protocol, in which the optimizer submits candidates and receives acknowledgment of validated results.

IV. Individual Layer Evaluation

A. Experimental Setup

All evaluations use the IBM two-qubit benchmark in Eq. (2), except the larger Heisenberg trace used to exercise numeric checking. Each layer evaluates the object relevant to its specifications. The logical study varies the controller's iteration bound and observable count. The numeric study verifies arithmetic kernels and checks stored term-level records. The probabilistic study measures the size and checking cost of Gaussian DTMCs over the finite grid. The runtime study measures monitoring cost and checks sampled simulator and hardware executions.

For logical verification, NuXmv evaluates the refined controller and its phase-reachability queries. Its budget terminal uses an abort label. Numeric checking uses version 2.0 of the trace checker on a two-qubit Estimator trace and an exact-estimate Heisenberg trace. Both record expectations and contributions, without raw measurement counts. These traces support checking the reported arithmetic and record structure.

The probabilistic scalability study fixes the circuit, grid, initial point, measurement groups, and shot allocation in Table II. It varies the horizon of a reachable, time-unrolled Gaussian DTMC. Success uses the specified ideal-energy gap. Model sizes are deterministic counts, construction has one timing measurement per horizon, and checking uses the median of three tool timings.

Runtime timing uses Qiskit's StatevectorSampler, four independent Pauli submissions per evaluation, and 1,024 shots per term. An ask/tell pattern-search optimizer uses an initial step of 0.4, a shrink factor of 0.5, and the recorded energy-stability rule: tolerance 0.005 for three successive checks after at least ten evaluations. The cost sweep fixes the workload seed at seven and evaluates counts from ten to fifty in increments of ten. It has 5 warm-ups and 30 measured repetitions per condition/count. Independent-condition bootstrap resampling gives uncertainty intervals for the medians and added time. The hardware assessment uses the recorded `ibm_marrakesh` execution and its simulator counterpart. Their results concern monitored conformance and declared termination.

Section V specifies the two-stage evaluation using the fixed configuration in Table II and repeated finite-shot executions. All reported numerical text, tables, and plots are generated from the corresponding source artifacts.

B. Logical and Numeric Outcomes

The refined controller has 41 true properties in its NuXmv output and 18 positive phase-reachability queries. For the iteration-bound/term-count pairs $(3, 2)$, $(5, 3)$, $(10, 5)$, $(20, 10)$, wall-clock measurements are respectively 0.04, 0.02, 0.03, 0.07 s. Every configuration returns the same 41 true properties and zero false properties. These are single local timing measurements of bounded controllers, so the observed costs apply to the tested configurations.

For the controller used in the two-stage evaluation, 7 logical properties hold. Three Dafny invocations report 24 verified units with zero errors. A verified unit is a tool-reported proof task. The specification mapping identifies how these tasks support N1–N4, and the result manifest records their distribution across modules.

The numeric checker accepts all 25 records of the 2-qubit, 4-term Estimator trace and all 60 records of the 14-qubit, 287-term exact-estimate Heisenberg trace. These results demonstrate arithmetic and record conformance for the supplied values, including a larger term list. Raw-count reconstruction requires measurement records beyond the fields available in either trace. Timing scaling and optimization success are outside this trace assessment. Synthetic fixtures remain classified as checker inputs.

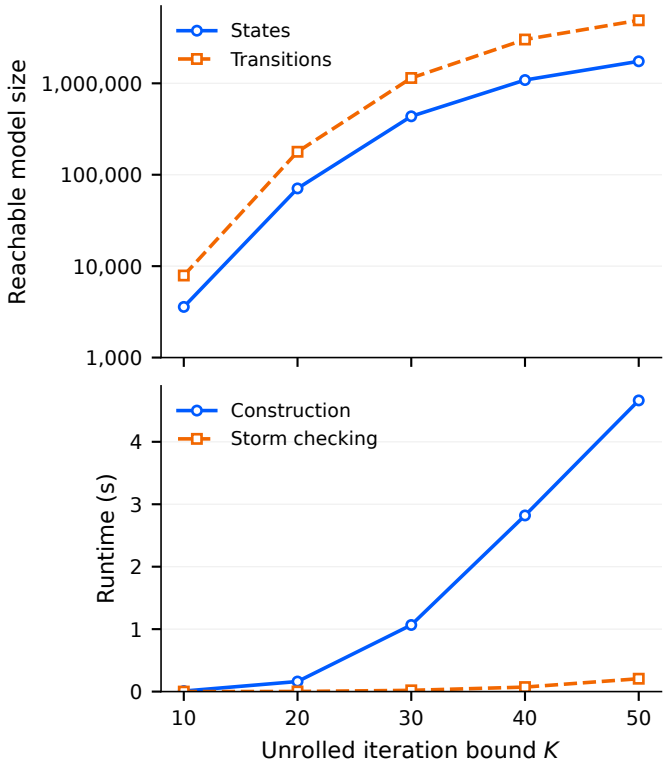


Fig. 2. Time-unrolled Gaussian DTMC scalability under Table II. Top: reachable states and transitions on a logarithmic scale. Bottom: one construction-time measurement and the median of three Storm checking times per iteration bound.

C. Probabilistic Model Size and Cost

The scalability study builds a reachable, time-unrolled DTMC for each iteration bound. It merges successful paths into an absorbing success state and exhausted paths into an absorbing timeout state. Figure 2 shows the model sizes and costs. At the largest bound, it contains 1,741,676 states and 4,889,851 transitions. The homogeneous grid/coordinate-phase DTMC used for the two-stage evaluation has 262,144 states and 755,176 transitions. Its fixed representation supports queries at different horizons, while time unrolling explicitly includes the iteration depth in the state space.

The cost panel separates construction from model checking. Construction excludes Gaussian-kernel preparation, and checking excludes parsing. A recorded zero checking time is below the tool's timing resolution. Source artifacts contain per-depth state counts, timings, the CSV summary, metadata, and generation source. The exported explicit scalability models are unavailable. The reported growth and costs can therefore be reconstructed from the recorded metadata and measurements, with direct model-file inspection available for the homogeneous DTMC.

The FakeJakarta density-matrix study records 347/400 successes (86.75 %). It assesses robustness under backend noise and compiled circuits. Those conditions extend beyond the ideal-Born assumptions of the Gaussian model, so the result

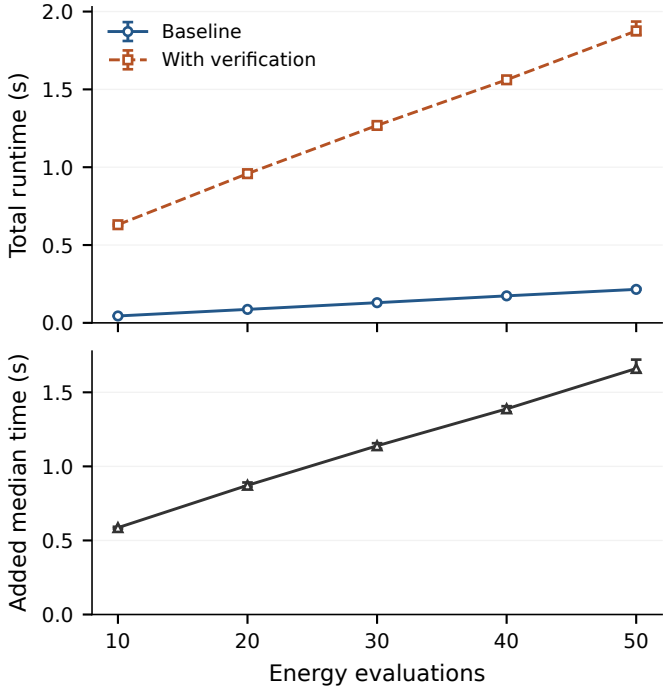


Fig. 3. Simulator runtime cost for a fixed workload seed. Top: baseline and verified medians. Bottom: their difference. Error bars are 95% percentile-bootstrap intervals from 30 measured runs per condition/count after 5 warm-ups.

describes noisy execution under its own configuration. The probability comparison in Section V uses ideal finite-shot runs and the model's success predicate.

D. Runtime Cost and Hardware Results

The runtime cost study has 50 warm-ups and 300 measured runs under the setup described above. Reusing the workload seed holds the sampled trajectory fixed while repetitions measure timing variation.

Figure 3 reports total runtime and the difference of condition medians. Verified timing includes VQE setup and sampling, monitor startup, synchronous checking and acknowledgments, trace and measurement-record persistence, and finalization. Baseline post-run serialization, fidelity analysis, and offline replay are excluded from the timing boundary. Bootstrap intervals use 10,000 independent-condition resamples of the measured times. The intervals describe uncertainty in the timing summaries for this fixed workload.

At the largest tested count, the baseline median is 0.215 s, the verified median is 1.876 s, and the added median time is 1.661 s (771.84 % relative increase). These measurements show substantial relative overhead on this fast simulator workload. Estimating overhead for the cyclic-grid executor would require repeated uninstrumented timings alongside its monitored executions. A systematic numeric-verification cost study is also unavailable.

Table III reports the sampled simulator and `ibm_marrakesh` executions and their online/replay

TABLE III
SIMULATOR AND HARDWARE RUNTIME RESULTS. "PASS/PASS" DENOTES ONLINE/REPLAY CONFORMANCE.

Execution	Evals.	Submissions	Shots	Online/replay
statevector_sampler	50	200	204,800	pass/pass
ibm_marrakesh	50	200	204,800	pass/pass

outcomes. Both terminate at the evaluation budget. Acceptance supports observed event ordering, arithmetic, resource accounting, and record completeness. These properties concern the ask/tell workflow under its simulator or hardware conditions. The Gaussian search prediction is assessed under the cyclic-grid configuration in Section V.

V. Cross-Layer Relationships and Agreement

A. Goals and Procedure

The two-stage evaluation examines whether the properties established before execution correspond to the requirements checked during execution, and whether their results agree within the stated assumptions. It fixes the circuit, operator, parameter grid, candidate rule, measurement groups, shot allocation, success predicate, and iteration bound in Table II. It also examines how controlled faults change the verification outcomes. The evaluated claim is consistency of the specified workflow across the two stages.

Stage 1 generates and checks the controller with NuXmv, verifies the numeric modules with Dafny, constructs the Gaussian kernel and DTMC, and runs Storm at the specified checkpoints. It also compiles the temporal monitor and records the results, configuration, and source hashes. Stage 2 checks that the execution uses this record, then performs 400 ideal-Born finite-shot runs with independently seeded count streams. It stores raw counts and all values consumed by the optimizer, checks events online, and replays the traces. Each run stops at first success or at $K = 50$ iterations. The ideal-energy gap defines success, as in the Gaussian model. Error closure handles online violations. No backend noise is introduced in this comparison.

For logical and numeric requirements, the comparison relates each prospective property result to the check on every observed run. The verdict categories defined in Section II distinguish satisfaction, violation, assumption mismatch, and unavailable checks. The meaning of each result follows its specification mapping. Numerical tolerances govern deterministic conformance. Stage 2 is gated on successful prospective verification, which restricts the evaluated combinations to executions authorized by Stage 1.

Probabilistic comparison operates on repeated executions. P1 compares the Gaussian reachability probability with the fraction reaching success by each checkpoint, including unsuccessful runs in the denominator. The probability at K is the primary comparison. Pointwise 95% Wilson intervals quantify binomial uncertainty [10]. P2 compares iterations

conditional on bounded success using the original 5,000-resample percentile-bootstrap interval. P3 applies the fixed shots per iteration to that quantity. A stochastic run that exhausts its budget is one permitted outcome of this bounded-success specification.

The supporting analysis computes two-sided score-test p-values at the nonzero-probability checkpoints and a normal-approximation test for P2. Holm adjustment [11] controls multiplicity for that family at $\alpha = 0.05$. The early zero-probability checkpoints are structurally uninformative, and P3 is derived from P2. This post-audit interpretation uses the existing observations and intervals. The comparison tests compatibility at the available sample size, without a preregistered equivalence margin.

B. Logical and Numeric Verification Agreement

Table IV gives the prospectively established property and runtime count for each logical and numeric requirement. The corresponding pre-execution and runtime results agree for all 400 baseline runs under the mapping's assumptions. The result establishes conformance of these observed executions. L4 agreement covers terminal absorption and bound conditions at both stages. The stronger termination-reason requirement is assessed at runtime.

Raw-count reconstruction covers 40,554 candidate evaluations and 121,662 group executions. The maximum observed energy reconstruction error is 5.542×10^{-17}, below the specified tolerance. Expectation reconstruction has zero observed error. Total physical consumption is 124,581,888 shots across 13,518 optimizer iterations. Python online and replay verdicts agree for every run, and all baseline runs pass the DejaVu temporal replay. These counts quantify the checked observations under the logical and numeric specifications.

C. Gaussian Prediction and Finite-Shot Execution

Figure 4 compares the Gaussian first-success curve with empirical execution using the same success predicate. Forward propagation of the Gaussian kernel produces the curve, and Storm independently checks the specified checkpoint probabilities. Their maximum recorded numerical difference is 1.110×10^{-16}. The maximum quadrature row-normalization correction is 5.090×10^{-8}. These quantities assess numerical processing of the constructed model. The empirical comparison assesses its prediction for finite-shot execution.

At the primary bound, 366/400 runs succeed and 34 terminate normally at budget. The empirical probability is 91.50 %, with Wilson interval [88.36 %, 93.85 %], compared with 92.80 % modeled probability. The absolute difference is 1.30 percentage points. Table V combines checkpoint predictions, empirical counts, and original intervals with the conditional-resource results. Deviations and the supporting statistical interpretation are discussed here. Agreement here denotes statistical compatibility at the tested resolution.

P2 is 33.111 modeled iterations conditional on success versus 32.290 empirical iterations, with the original percentile-bootstrap interval [31.579, 32.995]. Table V preserves that

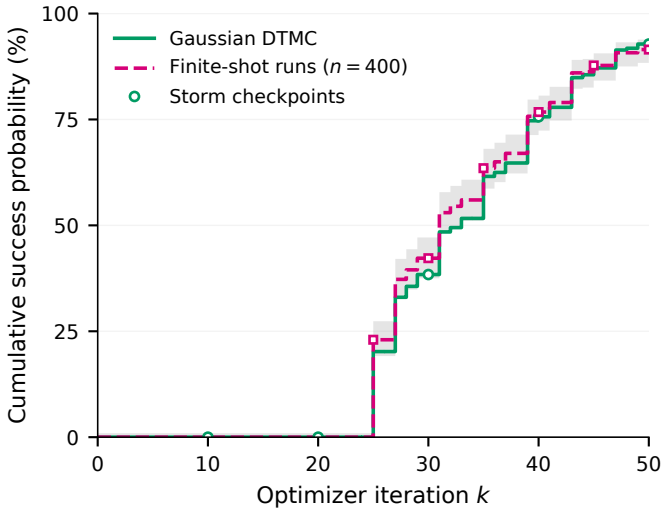


Fig. 4. Cumulative success probability for $E_{\text{ideal}}(z) - E_{\text{ref}} \leq 0.02$. Solid: Gaussian-DTMC propagation. Circles: Storm checkpoints. Dashed: 400 independently seeded ideal-Born finite-shot executions under Table II. Shading: pointwise 95% Wilson intervals.

interval, which does not contain the model mean. Marginal interval containment was the automated report's decision rule. The post-audit analysis interprets the supporting comparisons jointly: the raw P2 p-value is 0.024, becoming 0.094 after Holm adjustment. All informative comparisons are compatible at the family level. Thus P2 and its derived P3 result agree under the stated statistical comparison. The source report's original labels remain preserved for provenance. Statistical compatibility has the scope of this test and sample size, rather than a claim of distributional equivalence.

Including unsuccessful runs, mean expenditure is 33.795 iterations versus 34.327 modeled iterations. Total shot consumption includes budget-terminated paths, whereas P2 and P3 describe successful paths. The primary probability comparison's approximate normal-theory deviation detectable at 80% power is 3.62 percentage points at this sample size. This local planning approximation describes statistical resolution. An equivalence claim would require a declared margin and corresponding test. Supporting checkpoints are correlated, and the confidence band is pointwise.

D. Faults and Cross-Stage Discrepancies

Figure 5 reports the full logical and numeric verdict vector. The matrix shows which requirements respond to each fault, with distinct symbols for violations, changed assumptions, and unavailable checks. There are 5 fault scenarios, each repeated 10 times, for 50 tests. Execution faults use the first iteration/candidate boundary. The missing-count scenario deletes counts from a saved trace while preserving the corresponding control execution. It assesses how the checker handles incomplete records.

Premature commitment is caught by Python and DejaVu ordering checks. Altering the energy or coefficient pairing violates N2 while temporal properties pass, illustrating their

TABLE IV

LOGICAL AND NUMERIC VERIFICATION RESULTS. RUNTIME COUNTS GIVE AGREEING BASELINE RUNS. EACH CLASSIFICATION APPLIES UNDER THE PROSPECTIVE SCOPE AND ASSUMPTIONS IN TABLE I.

ID	Prospectively established scope	Runtime agreement	Cross-stage classification
L1	Group-completeness counter	400/400	Agreement under mapping
L2	Validation-readiness counter	400/400	Agreement under mapping
L3	Epoch and readiness reset	400/400	Agreement under mapping
L4	Terminal absorption and budget bounds	400/400	Agreement under mapping
N1	Normalization kernel with parity assumed	400/400	Agreement under mapping
N2	Real weighted sum and bound	400/400	Agreement under mapping
N3	List sum with group identity assumed	400/400	Agreement under mapping
N4	Wrap and integer tie-priority selection	400/400	Agreement under mapping

TABLE V

GAUSSIAN PREDICTIONS AND EMPIRICAL RESULTS FROM 400 EXECUTIONS. PROBABILITY INTERVALS ARE POINTWISE WILSON INTERVALS. RESOURCE INTERVALS ARE THE ORIGINAL PERCENTILE-BOOTSTRAP INTERVALS FOR 366 SUCCESSFUL RUNS. INTERVALS USE THE NOMINAL 95% LEVEL.

Quantity / unit	Gaussian model	Empirical result	95% interval
P1: bounded success probability (%)			
$k = 10$	0.00	0.00 (0/400)	[0.00, 0.95]
$k = 20$	0.00	0.00 (0/400)	[0.00, 0.95]
$k = 30$	38.39	42.25 (169/400)	[37.51, 47.14]
$k = 40$	75.63	76.75 (307/400)	[72.37, 80.62]
$k = 50$	92.80	91.50 (366/400)	[88.36, 93.85]
Resources conditional on bounded success			
P2: iterations	33.111	32.290	[31.579, 32.995]
P3: physical shots	305,152.7	297,581.1	[291,034.2, 304,077.6]

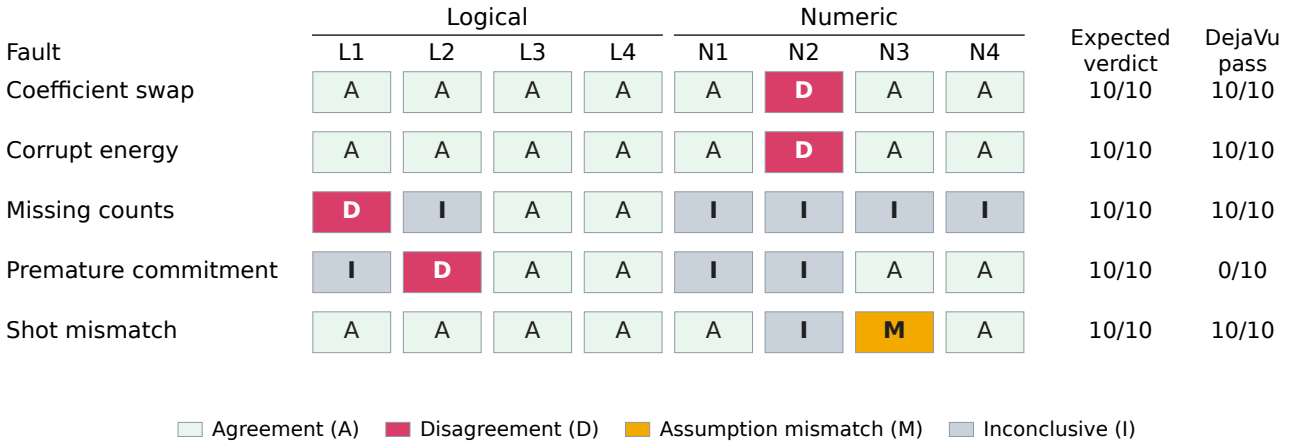


Fig. 5. Logical/numeric verdicts for 5 fault scenarios, each repeated 10 times. Every repetition in a row has the displayed vector. The legend defines the verdict symbols. Right-hand counts show the expected target verdict and acceptance by DejaVu temporal replay.

complementary scopes. A shot-allocation change produces an N3 assumption mismatch and makes the probabilistic comparison inapplicable. The injector configures this applicability label, so it records which stochastic assumptions the fault changes.

Missing counts make numeric reconstruction inconclusive. The checker also records an L1 disagreement because its completeness condition treats unavailable records as missing measurements. A deleted record alone does not establish that the original execution aggregated too early. The full vector exposes this classification limitation alongside the expected target detections. The suite covers the specified fault classes at one injection boundary. Terminal-boundary faults, stale inputs, and additional injection sites require further tests.

E. Summary of Verification Results

The evaluation begins with properties of the controller, arithmetic kernels, and Gaussian process, then checks the corresponding requirements on executions of the specified configuration. The mapping makes the role of each verification

layer explicit. Logical results describe allowed ordering, numeric results describe calculations under input assumptions, and probabilistic results describe bounded search. Runtime checking determines whether the logical and numeric requirements hold in each run, while repeated executions assess the probabilistic predictions.

The baseline agreement supports this connection at the level of the established properties. Logical and numeric checks accept every observed baseline run, and the bounded-success and conditional-resource comparisons are statistically compatible under the stated analysis. P3 follows P2 because shots per iteration are fixed. Successful comparisons therefore depend on the meaning and scope of the requirement, as well as on the reported verdict. L4 illustrates the distinction: both stages assess terminal closure and budget bounds, while correct termination reasons have runtime support.

Fault results show why the four aspects are useful together. Temporal checking detects premature commitment, whereas numeric reconstruction detects energy and coefficient errors that preserve temporal order. Changed shot allocations identify assumptions that must be restored before comparing stochastic outcomes. Missing counts reveal a further requirement for the implementation: it must distinguish incomplete observations from observed violations. Preserving the full verdict vector exposes this limitation.

The combined assessment thus provides traceable agreement among heterogeneous verification results within the tested configuration. Its practical value is the ability to locate the affected requirement and the limits of its support. Model growth and runtime costs in Section IV establish the measured expense of the corresponding verification components. Extending the conclusions requires stronger termination properties, broader fault coverage, and evaluations under larger or noisy execution models.

VI. Related Work

VQE and variational hybrid algorithms combine classical decisions with quantum measurements [1], [2]. Reviews of VQE examine measurement strategies, optimization, noise, and their impact on practical performance [3]. Direct-search literature provides the optimization context for choosing candidates using objective values [5]. Here, the finite-grid search makes these decisions explicit enough to model their probability and inspect their execution.

NuXmv, Dafny, Storm, and DejaVu provide established methods for model checking, deductive verification, probabilistic analysis, and temporal monitoring [6]–[9]. We use these methods according to the requirements of each layer. The contribution is a specification mapping and two-stage procedure that relates their results to an implemented VQE workflow and identifies the scope of agreement.

Statistical methods quantify the uncertainty in empirical comparison. Wilson intervals describe a binomial proportion [10], and Holm adjustment controls the family-wise rejection rate for a specified family of tests [11]. Their role is to assess compatibility between model predictions and sampled outcomes. The conditional-resource comparison illustrates why a marginal interval and a family-level interpretation answer different statistical questions.

VII. Discussion and Limitations

Verification scope. NuXmv is exhaustive for its encoded controller. Dafny proves kernels under mathematical semantics and preconditions. Storm is exhaustive for the constructed Gaussian DTMC. Runtime checking covers observed instrumented traces, and repeated executions provide statistical evidence. The framework relates these scoped results through specification mappings. Establishing a formal refinement from every model to the Python program remains future work.

Trusted base. Circuit and measurement-basis generation, bit order and parity interpretation, event instrumentation, proof tools, and the runtime reference checker are trusted. Hashes associate source and result files with the recorded experiment. Authenticating the physical origin of counts requires additional protection, particularly against consistent fabrication or relabeling. A certified bound on stochastic approximation error would also require analysis beyond the normal-CDF approximation and quadrature used here.

Generality and observability. The two-stage evaluation uses a small two-qubit benchmark with a fixed finite grid, ideal statistics, and an exact-energy stopping oracle. Its target, optimizer, grouped measurements, and finite budget define the evaluated boundary. The hardware execution and larger numeric trace demonstrate the corresponding runtime and arithmetic checks. Extending prospective agreement to those instances requires models and specifications for their configurations and observable stopping rules.

Statistical and audit boundaries. The P2/P3 interpretation is a post-audit reporting correction. Original marginal intervals and observations remain visible, and the sample size constrains statistical resolution. Development executions used the same seeds and are excluded from the reported sample counts. Their trajectories were already known before the recorded prospective verification, so this is an engineering evaluation rather than a blinded confirmatory study. The independent audit identified gaps in termination properties and classification of missing records. The reported scopes reflect these findings, with the experiment sources unchanged. The auditor's additional simulations support its statistical interpretation, but their saved arrays are unavailable. Their numerical summaries are therefore excluded. The reproducibility manifest also documents the unavailable scalability model exports.

Feedback scope. The framework detects inconsistencies and supports revising assumptions or specifications before regenerating the verification record and running the next execution. This feedback is manual. The evaluated workflow gates execution on successful prospective results, and the fault scenarios cover the specified boundary.

VIII. Conclusion

The four-layer architecture organizes VQE verification around logical, numeric, probabilistic, and runtime requirements. Its two-stage framework connects properties established

prospectively with the corresponding requirements checked during concrete execution of a specified configuration. The benchmark results support logical and numeric agreement within the verified scope and statistical compatibility of Gaussian predictions with repeated finite-shot executions. Fault vectors reveal complementary detection capabilities and a limitation in classifying missing records. Model-size and timing measurements quantify the cost of the evaluated components. Together, the results demonstrate a traceable assessment of heterogeneous verification methods. Future work will strengthen termination specifications, examine additional fault locations, and extend the models and runtime checks to larger instances and hardware noise.

Artifact Availability

The local reproducibility bundle includes this source, vector figures, editable architecture source, figure-generation scripts, generated numerical tables, `FIGURE_PLAN.md`, `FIGURE_MANIFEST.md`, and `PAPER_RESULT_TRACEABILITY.md`. The source manifest records hashes of the verification record, models, proof logs, raw traces, and individual-layer inputs. Original experimental artifacts and the reference manuscript are preserved. The bundle is currently a local research artifact.

References

[1] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, p. 4213, 2014.

[2] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, "The theory of variational hybrid quantum-classical algorithms," *New Journal of Physics*, vol. 18, no. 2, p. 023023, 2016.

[3] J. Tilly, H. Chen, S. Cao, D. Picozzi, K. Setia, Y. Li, E. Grant, L. Wossnig, I. Rungger, G. H. Booth, and J. Tennyson, "The variational quantum eigensolver: A review of methods and best practices," *Physics Reports*, vol. 986, pp. 1–128, 2022.

[4] IBM Quantum, "The variational quantum eigensolver (VQE)," Quantum Diagonalization Algorithms, IBM Quantum Learning, sec. 3, Example Hamiltonian; accessed October 6, 2026. [Online]. Available: https://quantum.cloud.ibm.com/learning/en/courses/quantum-diagonalization-algorithms/vqe

[5] T. G. Kolda, R. M. Lewis, and V. Torczon, "Optimization by direct search: New perspectives on some classical and modern methods," *SIAM Review*, vol. 45, no. 3, pp. 385–482, 2003.

[6] R. Cavada, A. Cimatti, M. Dorigatti, A. Griggio, A. Mariotti, A. Micheli, S. Mover, M. Roveri, and S. Tonetta, "The nuXmv symbolic model checker," in *Computer Aided Verification*. Springer, 2014, pp. 334–342.

[7] The Dafny Project, *Dafny Reference Manual*, version 4.11.0; accessed October 6, 2026. [Online]. Available: https://dafny.org/v4.11.0/DafnyRef/DafnyRef

[8] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann, and M. Volk, "The probabilistic model checker Storm," *International Journal on Software Tools for Technology Transfer*, vol. 24, no. 4, pp. 589–610, 2022.

[9] K. Havelund, D. Peled, and D. Ulus, "DejaVu: A monitoring tool for first-order temporal logic," 2018. [Online]. Available: https://www.havelund.com/Publications/dejavu-mtcps-2018.pdf

[10] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[11] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733